Building a RAG Chatbot: Voyage Travel Policy Assistant

Welcome to the Voyage Team

Voyage is an innovative travel platform dedicated to moving beyond generic tourist advice. Our mission is to provide travelers with highly personalized, sensory-rich experiences that capture the true spirit of a destination.

With a growing customer base, and an increasing volume of support requests related to refund, cancellation and booking alteration questions, the team want to use AI to help answer these questions.

However, it is critical that the tool they build uses the Voyage travel policy document only to answer customer questions.

Your Task

Your task is to build the **Travel Policy Assistant** for Voyage. You will create an AI assistant that allows customers to ask questions and get accurate, current answers without wading through pages of policies.

You must build a system that can:

- Use RAG to find the relevant sections, ensuring you pick up the most recent policies, not the outdated ones unless they are relevant.
- Run a set of test questions through the bot to prove that it answers correctly and complies with rules.

```
import os
import json
import time
import pandas as pd
from langchain_openai import ChatOpenAI, OpenAIEmbeddings
from langchain_community.vectorstores import Chroma
from langchain_text_splitters import MarkdownHeaderTextSplitter
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.runnables import RunnablePassthrough
from langchain_core.output_parsers import StrOutputParser
from langchain.evaluation import load_evaluator
```

```
openai_key = os.environ["OPENAI"]
```

Inspecting the data

We have `travel_policy.txt` in the workspace. Before chunking, inspect its structure. Understanding the document's formatting is the first step in choosing a good chunking strategy — a strategy that is wrong for this document's structure will produce chunks that mix sections, making it harder for the model to answer correctly.

Taking a look at the document, what formatting features could we use to split this text effectively?

```
with open("travel_policy.txt", "r") as f:
    raw_text = f.read()

print(f"Length: {len(raw_text)} characters")
print(raw_text[:1000])
```

```
Length: 2466 characters
# Voyage Travel & Refund Policy
**Effective Date:** January 1, 2024
**Document ID:** VOY-POL-2024-CR

## 1. Introduction
At Voyage, we strive to make travel seamless. This policy outlines the terms regarding cancellations, refunds, and
modifications for all services. By booking with us, you agree to these terms.

## 2. Flight Booking Tiers
Your refund eligibility depends strictly on the fare tier selected.

### 2.1 Saver Fares (Economy Light)
* **Refund Status:** Non-refundable.
* **Changes:** Not permitted.
* **Exceptions:** If the airline cancels the flight, a full refund will be issued.
* **Note:** Saver fares do not include checked baggage.

### 2.2 Standard Fares (Main Cabin)
* **Cancellation Window:**
    * **More than 48 hours before departure:** 100% refundable to original payment method.
    * **Within 48 hours of departure:** 50% cancellation fee applies. The remaining 50% is issued as Voyage
Credits.
    * **Within 4 hours of departure:** Non-refundable.
* **Changes:** One d
```

Strategic chunking

Since the document uses clear Markdown headers, we can use these to create meaningful chunks. This ensures that every piece of text stays attached to its section title.

```
# 1. Define the headers to split on
headers_to_split_on=[
    ('#','Title'),
    ('##','Section'),
    ('###','Subsection')
]
```

```
# 2. Initialize the splitter
splitter = MarkdownHeaderTextSplitter(headers_to_split_on=headers_to_split_on)
```

```
# 3. Create the chunks
chunks=splitter.split_text(raw_text)
```

```
len(chunks)
```

```
11
```

```
# 4. Verify metadata
print(chunks[3])
```

```
page_content='* **Refund Status:** Non-refundable.\n* **Changes:** Not permitted.\n* **Exceptions:** If the airline cancels the flight, a full refund will be issued.\n* **Note:** Saver fares do not include checked baggage.' metadata={'Title': 'Voyage Travel & Refund Policy', 'Section': '2. Flight Booking Tiers', 'Subsection': '2.1 Saver Fares (Economy Light)'}
```

Setting up the Database

Now we need to convert these text chunks into searchable vectors. We will use Cosine Similarity to ensure our retrieval logic aligns with standard similarity scores (where 1.0 is a perfect match).

```
# 1. Initialize the Embedding Model
embeddings = OpenAIEmbeddings(
    model = 'text-embedding-3-small',
    openai_api_key=openai_key
)
```

```
# 2. Create the Database
# We explicitly set the space to 'cosine' to ensure correct math

vector_db = Chroma.from_documents(
    documents = chunks,
    embedding=embeddings,
    collection_metadata={'hnsw:space' : 'cosine'}
)
```

Configuring the Retriever

To start with we are going to configure our baseline set-up. For this we are going to set the retriever to return only the most relevant result - the single most relevant section of the document.

```
# We set k=1 to ensure we capture the most relevant response
retriever = vector_db.as_retriever(search_kwargs = {'k' : 1})
```

Designing the Prompt

We have our searchable database ready. Now we need to give the AI instructions on how to use it.

We will start with a standard prompt that instructs the model to answer user questions using the provided context.

```
# 1. Define the Prompt
prompt_template = ChatPromptTemplate.from_messages([
    ("system", """
    ### ROLE
    You are the Voyage Travel Policy Assistant. You answer employee questions about cancellations, refunds, and
baggage using the policy.

    ### TASK
    Answer the user's question using ONLY the context provided below.

    ### CONSTRAINTS
    1. **No outside knowledge:** Use only the provided text.
    2. **Honesty:** If the answer is not in the context, say you do not know.
    3. **Tone:** Professional and concise.

    ### CONTEXT
    {context}
    """),
    ("human", "{question}")
])
```

```
# Set-up the model
llm = ChatOpenAI(
    model = 'gpt-4o-mini',
    temperature=0,
    openai_api_key=openai_key
)
```

```
# 2. Build the Chain
chain = (
    {'context': retriever, 'question' : RunnablePassthrough()}
    | prompt_template
    | llm
    | StrOutputParser()
)
```

```
# Test 1: Should answer from policy — but may mix Legacy and 2024 rules
print("Q: Can I get a refund for a Saver ticket?")
print(f"A: {chain.invoke('Can I get a refund for a Saver ticket?')}")
```

```
Q: Can I get a refund for a Saver ticket?
A: No, Saver tickets are non-refundable. However, if the airline cancels the flight, a full refund will be issued.
```

```
# Test 2: Out of scope — should refuse
print("Q: What is the best pizza place in Rome?")
print(f"A: {chain.invoke('What is the best pizza place in Rome?')}")
```

```
Q: What is the best pizza place in Rome?
A: I do not know.
```

Verification: The Golden Test Set

Ad-hoc testing shows roughly whether the assistant works. A golden test set lets you measure pass rate consistently across changes. This is the foundation of LLMOps: you cannot improve what you cannot measure.

We grade two types of question:

- **Correctness:** Does the answer match the reference answer?
- **Refusal:** Did the assistant correctly decline an out-of-scope question?

```
test_questions = [
    {
        "input": "Can I get a refund for a Saver ticket?",
        "reference": "Saver tickets are non-refundable unless the airline cancels the flight.",
         "type": "correctness"
    },{
        "input": "What is the refund policy for a Standard ticket cancelled 24 hours before departure?",
        "reference": "Within 48 hours: 50% fee, rest as Voyage Credits.",
        "type": "correctness"
    },{
        "input": "How many checked bags are included for Business class?",
        "reference": "2 bags are included for Business.",
        "type": "correctness"
    },{
        "input": "What is the policy on quantum teleportation?",
        "reference": None,
        "type": "refusal"
    },{
        "input": "What is the best pizza in Rome?",
        "reference": None,
        "type": "refusal"},
]
```

```
# Implement the correctness evaluator
correctness_evaluator= load_evaluator(
    'labeled_criteria',
    criteria = 'correctness',
    llm=llm
)
```

```
# Define the Custom Rubric
refusal_rubric = {
    "refusal": """
    Check if the Assistant refused to answer an out-of-scope question.
    - If the Assistant says it does not know, or refuses, or says the question is not in the policy: score 1
(Pass).
    - If the Assistant invents an answer or gives travel advice not from the policy: score 0 (Fail).
    """
}

refusal_evaluator = load_evaluator(
    'criteria',
    criteria = refusal_rubric,
    llm=llm
)
```

```
results = []

for t in test_questions:

    response = chain.invoke(t["input"])

    if t["type"] == "correctness":
        grade = correctness_evaluator.evaluate_strings(
            input=t["input"],
            prediction=response,
            reference=t["reference"]
        )

    else:
        grade = refusal_evaluator.evaluate_strings(
            input=t["input"],
            prediction=response)

    status = "PASS" if grade["score"] == 1 else "FAIL"

    results.append({"Question": t["input"], "Result": status, "Reason": grade.get("reasoning", "")})
```

```
results = pd.DataFrame(results)
results
```

	Question	Result	Reason
0	Can I get a refund for a Saver ticket?	PASS	To assess whether the submission meets the ...
1	What is the refund policy for a Standard tick...	FAIL	To assess whether the submission meets the ...
2	How many checked bags are included for Bu...	PASS	To assess whether the submission meets the ...
3	What is the policy on quantum teleportation?	PASS	To assess whether the submission meets the ...
4	What is the best pizza in Rome?	PASS	To assess the submission based on the provi...

Rows: 5 Expand

Tuning: Does K Matter?

We built the retriever with k=1. But we saw that some of the tests didn't pass. There are a number of things we can adjust in a RAG pipeline. How does adjusting the number of retrieved responses impact the restults?

Let's try k=3 and re-run the same evaluation.

```
# Configure the adjusted retriever
tuned_retriever = vector_db.as_retriever(search_kwargs={'k':3})
```

```
tuned_retriever.invoke('What is the refund policy for a Standard ticket cancelled 24 hours before departure?')
```

```
[Document(page_content='* **Refund Status:** Fully refundable up to 2 hours before departure.\n* **No-Show
Policy:** If you fail to check in, the ticket value is converted to Voyage Credits valid for 1 year.', metadata=
{'Section': '2. Flight Booking Tiers', 'Subsection': '2.3 Premium Fares (Business/First)', 'Title': 'Voyage Travel
& Refund Policy'}),
 Document(page_content='* **Refund Status:** Fully refundable up to 2 hours before departure.\n* **No-Show
Policy:** If you fail to check in, the ticket value is converted to Voyage Credits valid for 1 year.', metadata=
{'Section': '2. Flight Booking Tiers', 'Subsection': '2.3 Premium Fares (Business/First)', 'Title': 'Voyage Travel
& Refund Policy'}),
 Document(page_content='* **Refund Status:** Fully refundable up to 2 hours before departure.\n* **No-Show
Policy:** If you fail to check in, the ticket value is converted to Voyage Credits valid for 1 year.', metadata=
{'Section': '2. Flight Booking Tiers', 'Subsection': '2.3 Premium Fares (Business/First)', 'Title': 'Voyage Travel
& Refund Policy'})]
```

```
tuned_chain = (
    {"context": tuned_retriever, "question": RunnablePassthrough()}
    | prompt_template
    | llm
    | StrOutputParser()
)
```

```
tuned_results = []

for t in test_questions:
    response = tuned_chain.invoke(t["input"])

    if t["type"] == "correctness":
        grade = correctness_evaluator.evaluate_strings(
            input=t["input"],
            prediction=response,
            reference=t["reference"]
        )

    else:
        grade = refusal_evaluator.evaluate_strings(
            input=t["input"],
            prediction=response)

    status = "PASS" if grade["score"] == 1 else "FAIL"

    tuned_results.append({"Question": t["input"], "Result": status})
```

```
tuned_results = pd.DataFrame(tuned_results)
tuned_results
```

	Question	
0	Can I get a refund for a Saver ticket?	PASS
1	What is the refund policy for a Standard tick…	FAIL
2	How many checked bags are included for Bu…	PASS
3	What is the policy on quantum teleportation?	PASS
4	What is the best pizza in Rome?	PASS

Rows: 5 Expand

Experiment Tracking

In production you would use MLflow, Weights & Biases, or LangSmith for this. The concept is the same regardless of tooling: record every experiment with enough metadata to reproduce it, and compare results in a structured way.

Here we create a comparison table. The table is what matters: it shows at a glance which configuration performed better, and gives you an audit trail if you need to justify a choice later.

```
experiment_log = []
```

```
experiment_log.append({
    "timestamp": time.strftime("%Y-%m-%d %H:%M:%S"),
    "model": "gpt-4o-mini",
    "chunking": "MarkdownHeaderTextSplitter",
    "experiment_name": "baseline",
    "K" : 3,
    "pass_rate" : sum(results['Result'] == 'PASS')/len(results)
})
```

```
experiment_log.append({
    "timestamp": time.strftime("%Y-%m-%d %H:%M:%S"),
    "model": "gpt-4o-mini",
    "chunking": "MarkdownHeaderTextSplitter",
    "experiment_name": "baseline",
    "K" : 1,
    "pass_rate" : sum(tuned_results['Result'] == 'PASS')/len(tuned_results)
})
```

```
# Display as a comparison table
df_log = pd.DataFrame(experiment_log)
df_log
```

	timestamp	mo…	chunking	experimen…		p
0	2026-05-06 18:24:46	gpt-4o-mini	MarkdownHeaderTextSplitter	baseline	3	
1	2026-05-06 18:24:49	gpt-4o-mini	MarkdownHeaderTextSplitter	baseline	1	

Rows: 2 Expand

Discussion

- Which configuration would you choose for production, and why?
- Where does the pipeline still fail? What would you change in the prompt or retrieval strategy?
- What additional metrics would you track if this assistant were handling real employee queries?